Hidden Markov Models

Dr Aishwarya S., Assistant Professor of Mathematics, MIT, Manipal

Markov Processes (Markov Chains)

A stochastic process describing a sequence of possible events is said to be a Markov Process (or a Markov Chain) if the probability of each event depends only on the state attained in the previous event.

The Markov property is also sometimes stated as follows: The future state depends only on the present state and not on the past states.

Mathematically, if $q_1, q_2, \dots, q_k$ is a sequence of state variables, then the Markov property is the following:

$$P(q_i = a \mid q_1, q_2, \dots, q_{i-1}) = P(q_i = a \mid q_{i-1}).$$

Formally, a Markov chain is specified by the following components:

- $Q = \{q_1, q_2, \ldots, q_N\}$, a set of N **States**.

- An $N \times N$ matrix A, called the **Transition Probability Matrix**, where the ij-th entry a_{ij} represents the probability of moving from state i to state j. Also, $\sum_{j=1}^{N} a_{ij} = 1$.

- An **Initial Probability Distribution** over the states, $\pi = [\pi_1, \pi_2, \ldots, \pi_N]$, where π_i is the probability that the Markov chain starts in state i. Also, $\sum_{i=1}^{N} \pi_i = 1$.

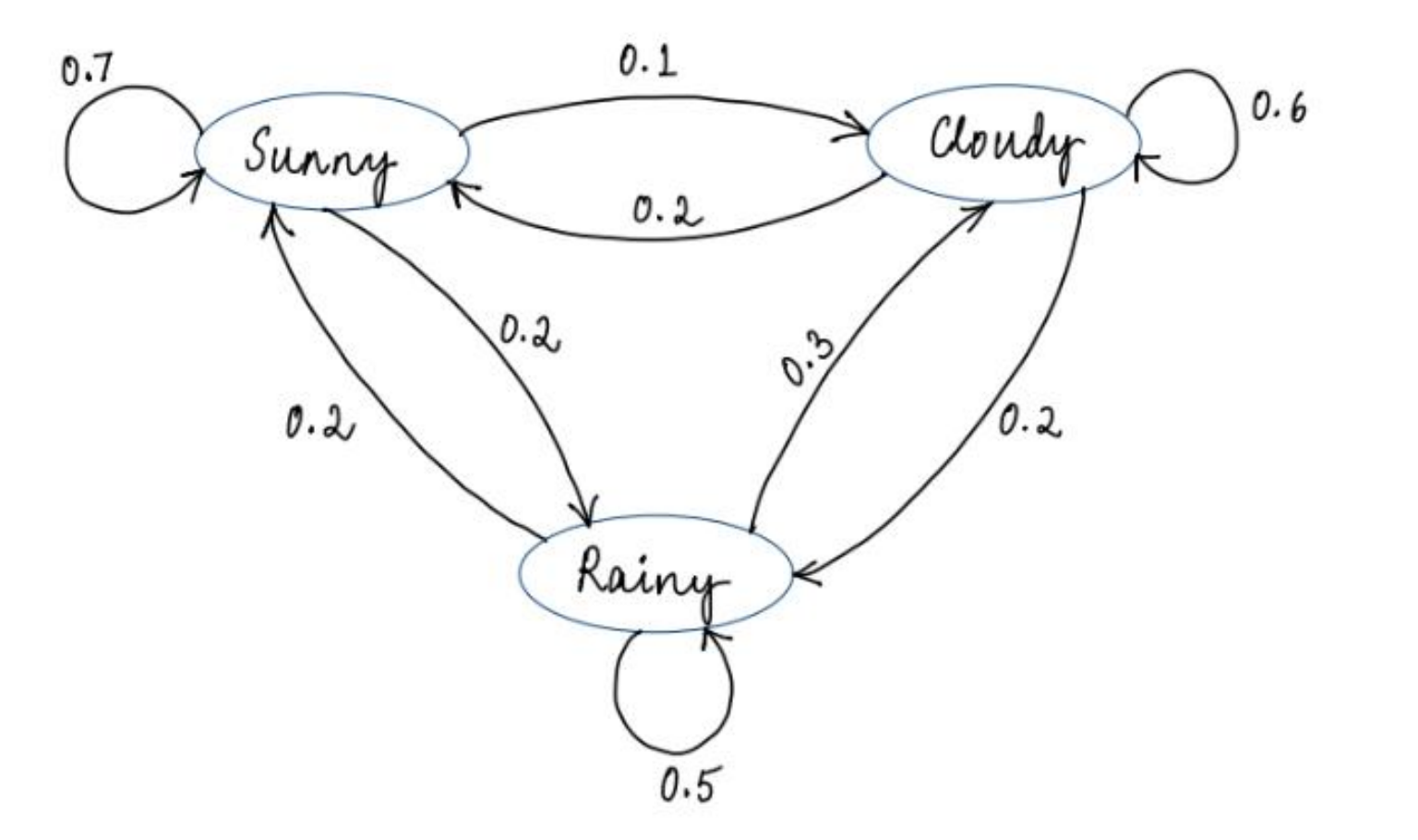
Example: A Markov chain for weather on a sequence of days (with possible states *Sunny*, *Cloudy*, *Rainy*).

Setting $\pi = [0.7 \quad 0.2 \quad 0.1]$ means that the probability of the weather on first day being *Sunny* is 0.7, being *Cloudy* is 0.2, and being *Rainy* is 0.1.

Suppose the transition probability matrix is as follows:

	Sunny	**Cloudy**	**Rainy**
Sunny	0.7	0.1	0.2
Cloudy	0.2	0.6	0.2
Rainy	0.2	0.3	0.5

The corresponding transition probability diagram is the directed graph shown below:



- **Markov chains** model systems where the current state depends only on the previous state; and they assume that these states are directly observable.

- However, in many real-world scenarios—such as speech recognition, bioinformatics, or financial modeling—the underlying states are **hidden**, and we only observe outputs that are probabilistically related to these states.

- **Hidden Markov Models (HMMs)** extend Markov chains by introducing this layer of hidden states, enabling us to infer the most likely sequence of states from observed data and capture complex temporal patterns.

Example:

In speech recognition, The **hidden variables** are the actual words spoken by the person. These are in the speaker's mind and vocal process. These are "hidden" because we cannot see or know these words directly.

The **observed variables** are the (possibly noisy) audio signals or features we record, like waveforms or frequency patterns. These are what the computer sees and analyzes.

The challenge: From the noisy audio signals (observed), we want to guess the actual sequence of words (hidden) that produced them. This is exactly what Hidden Markov Models (HMMs) help us to do.

A Hidden Markov Model (HMM) is specified by the following components:

- $Q = \{q_1, q_2, \ldots, q_N\}$, a set of N **States**.
- An $N \times N$ matrix A, called the **Transition Probability Matrix**, where the ij-th entry a_{ij} represents the probability of moving from state i to state j. Also, $\sum_{j=1}^{N} a_{ij} = 1$.
- $O = \{o_1, o_2, \ldots, o_M\}$, a set of M possible **Observations**.
- An $N \times M$ matrix B, called the **Emission Probability Matrix**, where the ij-th entry b_{ij} represents the probability of observation j being generated from state i. Also, $\sum_{j=1}^{M} b_{ij} = 1$.
- An **Initial Probability Distribution** over the states, $\pi = [\pi_1, \pi_2, \ldots, \pi_N]$, where π_i is the probability that the Markov chain starts in state i. Also, $\sum_{i=1}^{N} \pi_i = 1$.

Let Q_t be the random variable corresponding to the state at time t.
Let O_t be the random variable corresponding to the observation at time t.

Then the components of the HMM can be written explicitly as follows:

In the initial probability distribution $\pi = [\pi_1, \pi_2, \ldots, \pi_N]$, for each i,

$$\pi_i = P(Q_1 = i).$$

In the transition probability matrix $A = \left(a_{ij}\right)_{N \times N}$,

$$a_{ij} = P(Q_{t+1} = j \mid Q_t = i).$$

In the emission probability matrix $= B = \left(b_{ij}\right)_{N \times M}$,

$$b_{ij} = P(O_t = j \mid Q_t = i).$$

- b_{ij} is also denoted as $b_i(o_t)$.

Example:

Consider three possible weather states: *Sunny*, *Cloudy* and *Rainy* **[Hidden states]**.
Based on the weather, a person's mood may be either *Happy* or *Sad* **[Observed states]**.
We have the following components in the HMM:

$Q = \{Sunny, Cloudy, Rainy\}$

$O = \{Happy, Sad\}$

Initial probability distribution:
$\pi = [P(Q_1 = Sunny) \quad P(Q_1 = Cloudy) \quad P(Q_1 = Rainy)] = [0.7 \quad 0.2 \quad 0.1]$

Transition Probability Matrix: $a_{ij} = P(Q_{t+1} = j \mid Q_t = i)$ $(1 \leq i, j \leq 3, t \geq 1)$.

	Sunny	**Cloudy**	**Rainy**
Sunny	0.7	0.1	0.2
Cloudy	0.2	0.6	0.2
Rainy	0.2	0.3	0.5

Emission Probability Matrix: $b_{ij} = P(O_t = j \mid Q_t = i)$ $(1 \leq i \leq 3, 1 \leq j \leq 2, t \geq 1)$.

	Happy	**Sad**
Sunny	0.9	0.1
Cloudy	0.3	0.7
Rainy	0.6	0.4

What are the things we can do with HMMs?

- In his lectures at Bell Laboratories during 1960s, Jack Ferguson of the Institute for Defense Analysis (IDA) introduced the idea of characterizing the theoretical aspects of hidden Markov models in terms of solving three fundamental problems.

- Later, an influential tutorial by Rabiner (1989)[1], inspired by Jack Ferguson's lectures, established that hidden Markov models can be understood through three fundamental problems - a framework that shaped much of the research and applications that followed.

1. Rabiner, L. R. 1989. A tutorial on hidden Markov models and selected applications in speech recognition. Proceedings of the IEEE, 77(2):257–286.

The Three Fundamental HMM Problems

1. **Evaluation:** Given the HMM $\lambda = (A, B, \pi)$ and an observation sequence $O = o_1, o_2, \ldots, o_T$, compute $P(O \mid \lambda)$, that is, the probability of the observation sequence O given the model λ.

2. **Decoding:** Given the HMM $\lambda = (A, B, \pi)$ and an observation sequence $O = o_1, o_2, \ldots, o_T$, determine the hidden state sequence $Q = q_1, q_2, \ldots, q_T$ that best explains the observations (that is most likely to have generated O), that is, find $\text{argmax}_Q \, P(Q \mid O)$.

3. **Learning:** Given an observation sequence O and the set of states in the HMM, learn the HMM parameters $\lambda = (A, B, \pi)$, that is, find $\text{argmax}_\lambda \, P(O \mid \lambda)$.

1. Evaluation

Given the HMM $\lambda = (A, B, \pi)$ and an observation sequence $O = o_1, o_2, \ldots, o_T$, compute $P(O \mid \lambda)$, that is, the probability of the observation sequence O given the model λ.

Example: In the weather-mood HMM, what is the probability of observing the mood sequence *Happy*, *Sad*, *Happy* ?

A simpler situation:

Suppose we knew the weather on each day. For a given state sequence (e.g., *Sunny*, *Cloudy*, *Sunny*), we can compute the probability of mood sequence *Happy*, *Sad*, *Happy* as follows:

$$P(Happy, Sad, Happy \mid Sunny, Cloudy, Sunny)$$

$$= P(Happy \mid Sunny)\, P(Sad \mid Cloudy)\, P(Happy \mid Sunny)$$

$$= (0.9)(0.4)(0.9)$$

$$= 0.324.$$

In general, given a particular state sequence $Q = q_1, q_2, \ldots, q_T$ and an observation sequence $\mathrm{O} = o_1, o_2, \ldots, o_T$, the probability of the observation sequence is

$$P(O \mid Q) = \prod_{i=1}^{T} P(o_i|q_i).$$

Actual situation:

We do not actually know the weather sequence. Hence, we will need to compute the probability of the mood sequence *Happy*, *Sad*, *Happy* by summing over all possible weather sequences in the following way:

$$P(O) = \sum_{Q} P(O, Q)$$

$$= \sum_{Q} P(O \mid Q)\, P(Q).$$

For a particular state sequence Q, we have

$$P(O \mid Q)\, P(Q) = \prod_{i=1}^{T} P(o_i|q_i) \prod_{i=1}^{T} P(q_i|q_{i-1}).$$

Therefore,

$$P(O) = \sum_{Q=q_1,q_2,\ldots,q_T} \left(\prod_{i=1}^{T} P(o_i|q_i) \prod_{i=1}^{T} P(q_i|q_{i-1}) \right).$$

For an HMM with N hidden states and an observation sequence of T observations, there are N^T possible hidden sequences.

When N and T are both large, the number N^T is very large. Hence a brute-force search is inefficient (takes exponential time).

Instead of using the above exponential algorithm, a more efficient algorithm called **Forward Algorithm** is used.

It has time complexity $O(N^2T)$.

We want to calculate $P(O) = P(o_1, o_2, \dots, o_T)$.

We have

$$P(O) = P(o_1, o_2, \dots, o_T) = \sum_{j=1}^{N} P(o_1, o_2, \dots, o_T, Q_T = j).$$

For $1 \leq t \leq T$ and $1 \leq j \leq N$, denote $P(o_1, o_2, \dots, o_t, Q_t = j)$ by $\alpha_t(j)$.

Then

$$P(O) = P(o_1, o_2, \dots, o_T) = \sum_{j=1}^{N} \alpha_T(j).$$

For $2 \leq t \leq T$, we have

$$\alpha_t(j) = P(o_1, o_2, \dots, o_t, Q_t = j)$$

$$= \sum_{i=1}^{N} P(o_1, o_2, \dots, o_{t-1}, Q_{t-1} = i)\, P(Q_t = j \mid Q_{t-1} = i)\, P(o_t \mid Q_t = j)$$

$$= \sum_{i=1}^{N} \alpha_{t-1}(i)\, a_{ij}\, b_j(o_t)$$

which is a recursive equation for $\alpha_t(j)$.

For $t = 1$,

$$\begin{aligned}\alpha_1(j) &= P(o_1, Q_1 = j) \\ &= P(o_1 \mid Q_1 = j)\, P(Q_1 = j) \\ &= b_j(o_1)\, \pi_j\ .\end{aligned}$$

The forward algorithm is a dynamic programming technique. It uses the computations shown above to build a table of intermediate values and efficiently compute the probability of an observation sequence.

The Forward Algorithm

function FORWARD(*observations* of len T, *state-graph* of len N) **returns** *forward-prob*

create a probability matrix *forward[N,T]*
for each state s **from** 1 **to** N **do** ; initialization step
$forward[s,1] \leftarrow \pi_s * b_s(o_1)$
for each time step t **from** 2 **to** T **do** ; recursion step
for each state s **from** 1 **to** N **do**

$$forward[s,t] \leftarrow \sum_{s'=1}^{N} forward[s',t-1] * a_{s',s} * b_s(o_t)$$

$$forwardprob \leftarrow \sum_{s=1}^{N} forward[s,T]$$; termination step

return *forwardprob*

***Here, *forward[s,t]* represents $\alpha_t(s)$.**

Source: Daniel Jurafsky and James H. Martin, "Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models" (3rd edition), 2025. (Online manuscript released August 24, 2025.)
URL: https://web.stanford.edu/~jurafsky/slp3/

Steps:

1. Initialization:

$$\alpha_1(j) = \pi_j b_j(o_1) \quad 1 \leq j \leq N$$

2. Recursion:

$$\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t); \quad 1 \leq j \leq N, 1 < t \leq T$$

3. Termination:

$$P(O|\lambda) = \sum_{i=1}^{N} \alpha_T(i)$$

Source: Daniel Jurafsky and James H. Martin, "Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models" (3rd edition), 2025. (Online manuscript released August 24, 2025.)
URL: https://web.stanford.edu/~jurafsky/slp3/

Example:
Compute the probability of the observation sequence $P(Happy,\ Sad,\ Happy)$ using Forward Algorithm in the above example of HMM.

Given: $O = o_1, o_2, o_3 = Happy,\ Sad,\ Happy$.
We need to find $P(O)$.

Initialization :

$$\alpha_1(\text{Sunny}) = \pi_{\text{Sunny}} \times b_{\text{Sunny}}(\text{Happy})$$

$$= 0.7 \times 0.9$$

$$= 0.63$$

$$\alpha_1(\text{Cloudy}) = \pi_{\text{Cloudy}} \times b_{\text{Cloudy}}(\text{Happy})$$

$$= 0.2 \times 0.3$$

$$= 0.06$$

$$\alpha_1(\text{Rainy}) = \pi_{\text{Rainy}} \times b_{\text{Rainy}}(\text{Happy})$$

$$= 0.1 \times 0.6$$

$$= 0.06$$

Recursion :

$t = 2$:

$$\alpha_2(\text{Sunny}) = \sum_i \alpha_1(i) \cdot a_{i,\text{Sunny}} \cdot b_{\text{Sunny}}(\text{Sad})$$

$$= \alpha_1(\text{Sunny}) \cdot a_{\text{Sunny,Sunny}} \cdot b_{\text{Sunny}}(\text{Sad})$$

$$+ \alpha_1(\text{Cloudy}) \cdot a_{\text{Cloudy,Sunny}} \cdot b_{\text{Sunny}}(\text{Sad})$$

$$+ \alpha_1(\text{Rainy}) \cdot a_{\text{Rainy,Sunny}} \cdot b_{\text{Sunny}}(\text{Sad})$$

$$= (0.63 \times 0.7 \times 0.1) + (0.06 \times 0.2 \times 0.1) + (0.06 \times 0.2 \times 0.1)$$

$$= 0.0465.$$

$$\alpha_2(\text{Cloudy}) = \sum_i \alpha_1(i) \cdot a_{i,\text{Cloudy}} \cdot b_{\text{Cloudy}}(\text{Sad})$$

$$= \alpha_1(\text{Sunny}) \cdot a_{\text{Sunny,Cloudy}} \cdot b_{\text{Cloudy}}(\text{Sad})$$

$$+ \alpha_1(\text{Cloudy}) \cdot a_{\text{Cloudy,Cloudy}} \cdot b_{\text{Cloudy}}(\text{Sad})$$

$$+ \alpha_1(\text{Rainy}) \cdot a_{\text{Rainy,Cloudy}} \cdot b_{\text{Cloudy}}(\text{Sad})$$

$$= (0.63 \times 0.1 \times 0.7) + (0.06 \times 0.6 \times 0.7)$$

$$+ (0.06 \times 0.3 \times 0.7)$$

$$= 0.0819$$

$$\alpha_2(\text{Rainy}) = \sum_i \alpha_1(i) \cdot a_{i,\text{Rainy}} \cdot b_{\text{Rainy}}(\text{Sad})$$

$$= \alpha_1(\text{Sunny}) \cdot a_{\text{Sunny},\text{Rainy}} \cdot b_{\text{Rainy}}(\text{Sad})$$

$$+ \alpha_1(\text{Cloudy}) \cdot a_{\text{Cloudy},\text{Rainy}} \cdot b_{\text{Rainy}}(\text{Sad})$$

$$+ \alpha_1(\text{Rainy}) \cdot a_{\text{Rainy},\text{Rainy}} \cdot b_{\text{Rainy}}(\text{Sad})$$

$$= (0.63 \times 0.2 \times 0.4) + (0.06 \times 0.2 \times 0.4)$$

$$+ (0.06 \times 0.5 \times 0.4)$$

$$= 0.0672.$$

$t = 3 = T$:

$$\alpha_3(\text{Sunny}) = \sum_i \alpha_2(i) \cdot a_{i,\text{Sunny}} \cdot b_{\text{Sunny}}(\text{Happy})$$

$$= \alpha_2(\text{Sunny}) \cdot a_{\text{Sunny,Sunny}} \cdot b_{\text{Sunny}}(\text{Happy})$$
$$+ \alpha_2(\text{Cloudy}) \cdot a_{\text{Cloudy,Sunny}} \cdot b_{\text{Sunny}}(\text{Happy})$$
$$+ \alpha_2(\text{Rainy}) \cdot a_{\text{Rainy,Sunny}} \cdot b_{\text{Sunny}}(\text{Happy})$$

$$= (0.0465 \times 0.7 \times 0.9) + (0.0819 \times 0.2 \times 0.9)$$
$$+ (0.0672 \times 0.2 \times 0.9)$$

$$= 0.0561$$

$$\alpha_3(\text{Cloudy}) = \sum_i \alpha_2(i) \cdot a_{i,\text{Cloudy}} \cdot b_{\text{Cloudy}}(\text{Happy})$$

$$= \alpha_2(\text{Sunny}) \cdot a_{\text{Sunny},\text{Cloudy}} \cdot b_{\text{Cloudy}}(\text{Happy})$$

$$+ \alpha_2(\text{Cloudy}) \cdot a_{\text{Cloudy},\text{Cloudy}} \cdot b_{\text{Cloudy}}(\text{Happy})$$

$$+ \alpha_2(\text{Rainy}) \cdot a_{\text{Rainy},\text{Cloudy}} \cdot b_{\text{Cloudy}}(\text{Happy})$$

$$= (0.0465 \times 0.1 \times 0.3) + (0.0819 \times 0.6 \times 0.3)$$

$$+ (0.0672 \times 0.3 \times 0.3)$$

$$= 0.0222 .$$

$$\alpha_3(\text{Rainy}) = \sum_i \alpha_2(i) \cdot a_{i,\text{Rainy}} \cdot b_{\text{Rainy}}(\text{Happy})$$

$$= \alpha_2(\text{Sunny}) \cdot a_{\text{Sunny, Rainy}} \cdot b_{\text{Rainy}}(\text{Happy})$$

$$+ \alpha_2(\text{Cloudy}) \cdot a_{\text{Cloudy, Rainy}} \cdot b_{\text{Rainy}}(\text{Happy})$$

$$+ \alpha_2(\text{Rainy}) \cdot a_{\text{Rainy, Rainy}} \cdot b_{\text{Rainy}}(\text{Happy})$$

$$= (0.0465 \times 0.2 \times 0.6) + (0.0819 \times 0.2 \times 0.6)$$

$$+ (0.0672 \times 0.5 \times 0.6)$$

$$= 0.0356.$$

Termination :

$$P(O) = \alpha_3(\text{Sunny}) + \alpha_3(\text{Cloudy}) + \alpha_3(\text{Rainy}).$$

$$= 0.0561 + 0.0222 + 0.0356$$

$$= 0.1139 .$$

2. Decoding

Given the HMM $\lambda = (A, B, \pi)$ and an observation sequence $O = o_1, o_2, \ldots, o_T$, determine the hidden state sequence $Q = q_1, q_2, \ldots, q_T$ that best explains the observations (that is most likely to have generated O), that is, find $\text{argmax}_Q \, P(Q \mid O)$.

Example: In the weather-mood HMM, what is the hidden state sequence that is most likely to have generated the mood sequence *Happy*, *Sad*, *Happy* ?

For an HMM with N hidden states and an observation sequence of T observations, there are N^T possible hidden sequences.

Hence a brute-force search is inefficient (takes exponential time).

Instead of using the brute-force algorithm, a more efficient algorithm called **Viterbi Algorithm** is used.

It has time complexity $O(N^2T)$.

Viterbi Algorithm

function VITERBI(*observations* of len T,*state-graph* of len N) **returns** *best-path*, *path-prob*

create a path probability matrix *viterbi*$[N,T]$
for each state s **from** 1 **to** N **do** ; initialization step
    $viterbi[s,1] \leftarrow \pi_s * b_s(o_1)$
    $backpointer[s,1] \leftarrow 0$
for each time step t **from** 2 **to** T **do** ; recursion step
  for each state s **from** 1 **to** N **do**
    $viterbi[s,t] \leftarrow \max_{s'=1}^{N} viterbi[s',t-1] * a_{s',s} * b_s(o_t)$
    $backpointer[s,t] \leftarrow \operatorname{argmax}_{s'=1}^{N} viterbi[s',t-1] * a_{s',s} * b_s(o_t)$
$bestpathprob \leftarrow \max_{s=1}^{N} viterbi[s,T]$; termination step
$bestpathpointer \leftarrow \operatorname{argmax}_{s=1}^{N} viterbi[s,T]$; termination step
$bestpath \leftarrow$ the path starting at state *bestpathpointer*, that follows backpointer[] to states back in time
return *bestpath*, *bestpathprob*

Source: Daniel Jurafsky and James H. Martin, "Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models" (3rd edition), 2025. (Online manuscript released August 24, 2025.)
URL: https://web.stanford.edu/~jurafsky/slp3/

Steps:

1. **Initialization:**

$$v_1(j) = \pi_j b_j(o_1) \qquad 1 \le j \le N$$
$$bt_1(j) = 0 \qquad 1 \le j \le N$$

2. **Recursion**

$$v_t(j) = \max_{i=1}^{N} v_{t-1}(i)\, a_{ij}\, b_j(o_t); \quad 1 \le j \le N, 1 < t \le T$$
$$bt_t(j) = \operatorname*{argmax}_{i=1}^{N} v_{t-1}(i)\, a_{ij}\, b_j(o_t); \quad 1 \le j \le N, 1 < t \le T$$

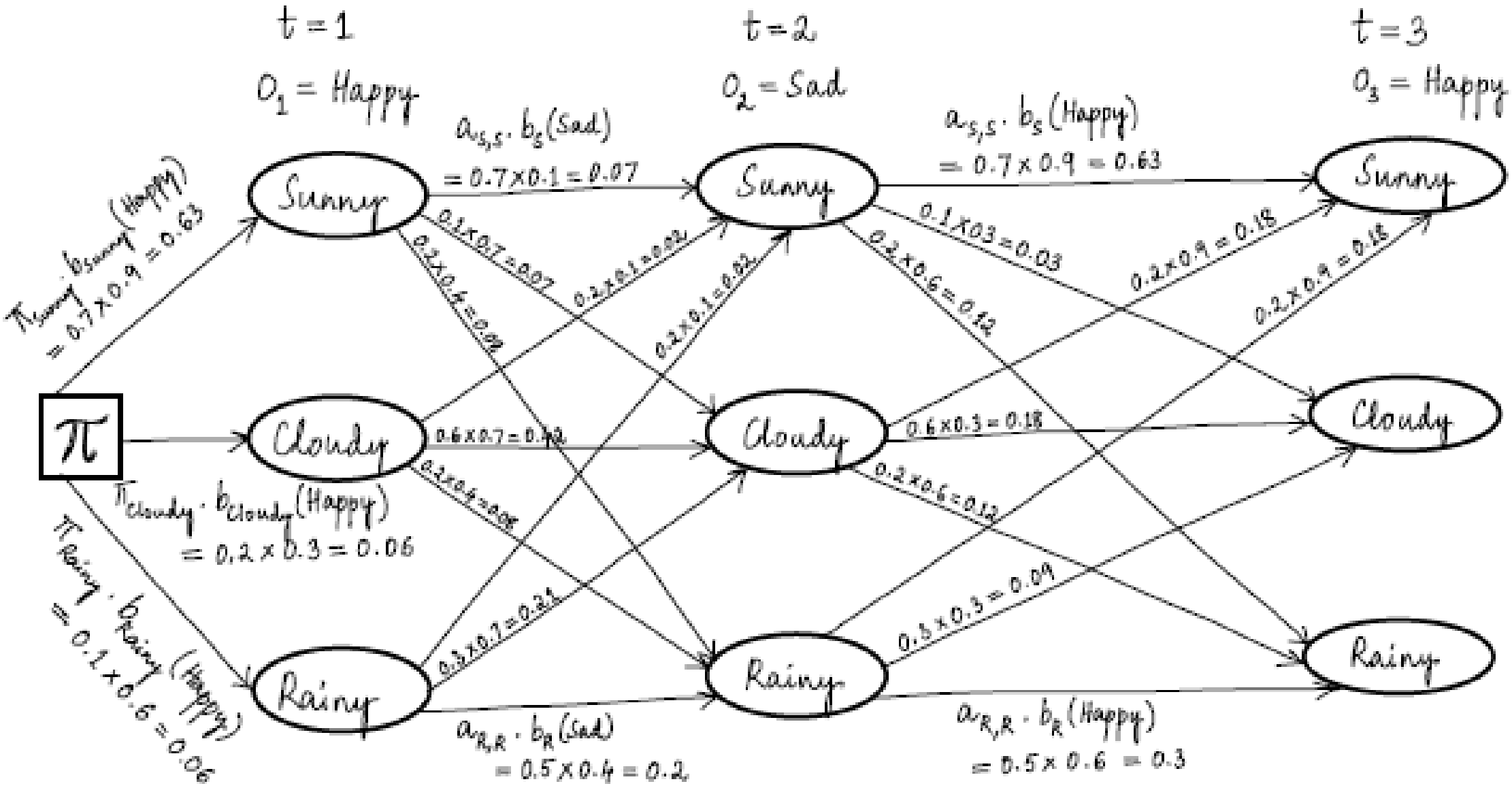
3. **Termination:**

$$\text{The best score:} \quad P* = \max_{i=1}^{N} v_T(i)$$
$$\text{The start of backtrace:} \quad q_T* = \operatorname*{argmax}_{i=1}^{N} v_T(i)$$

Source: Daniel Jurafsky and James H. Martin, "Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models" (3rd edition), 2025. (Online manuscript released August 24, 2025.)
URL: https://web.stanford.edu/~jurafsky/slp3/

Example:
Find the hidden state sequence that is most likely to have generated the mood sequence *Happy*, *Sad*, *Happy* using Viterbi Algorithm in the above example of HMM.

Given: $O = o_1, o_2, o_3 = \textit{Happy, Sad, Happy}$.
We need to find $\text{argmax}_Q \, P(Q = q_1, q_2, q_3 \mid O)$.


t = 1
O1 = Happy
t = 2
O2 = Sad
t = 3
O3 = Happy
π
Sunny
Cloudy
Rainy
Sunny
Cloudy
Rainy
Sunny
Cloudy
Rainy
π_Sunny · b_Sunny(Happy) = 0.7 × 0.9 = 0.63
π_Cloudy · b_Cloudy(Happy) = 0.2 × 0.3 = 0.06
π_Rainy · b_Rainy(Happy) = 0.1 × 0.6 = 0.06
a_S,S · b_S(Sad) = 0.7 × 0.1 = 0.07
a_S,S · b_S(Happy) = 0.7 × 0.9 = 0.63
a_R,R · b_R(Sad) = 0.5 × 0.4 = 0.2
a_R,R · b_R(Happy) = 0.5 × 0.6 = 0.3
0.1 × 0.7 = 0.07
0.2 × 0.6 = 0.12
0.2 × 0.1 = 0.02
0.2 × 0.1 = 0.02
0.6 × 0.7 = 0.42
0.2 × 0.4 = 0.08
0.3 × 0.7 = 0.21
0.1 × 0.3 = 0.03
0.2 × 0.9 = 0.18
0.2 × 0.9 = 0.18
0.6 × 0.3 = 0.18
0.2 × 0.6 = 0.12
0.5 × 0.3 = 0.09

$$V_2(S) = \max\{0.63 \times 0.07,\ 0.06 \times 0.02,\ 0.06 \times 0.02\} \qquad bt_2(S) = S$$
$$= 0.0441.$$

$$V_2(C) = \max\{0.63 \times 0.07,\ 0.06 \times 0.42,\ 0.06 \times 0.21\} \qquad bt_2(C) = S$$
$$= 0.0441.$$

$$V_2(R) = \max\{0.63 \times 0.08,\ 0.06 \times 0.08,\ 0.06 \times 0.2\} \qquad bt_2(R) = S$$
$$= 0.0504.$$

$$v_3(S) = \max\{0.0441 \times 0.63, 0.0441 \times 0.18, 0.0504 \times 0.18\} \qquad bt_3(S) = S$$
$$= 0.0278.$$

$$v_3(C) = \max\{0.0441 \times 0.03, 0.0441 \times 0.18, 0.0504 \times 0.09\} \qquad bt_3(C) = C$$
$$= 0.0079.$$

$$v_3(R) = \max\{0.0441 \times 0.12, 0.0441 \times 0.12, 0.0504 \times 0.3\} \qquad bt_3(R) = R$$
$$= 0.01512.$$

$$p^* = \max\{0.0278, 0.0079, 0.01512\}$$
$$= 0.0278.$$

$$q_T^* = S.$$

Hence, given the observation sequence $O = Happy, Sad, Happy$,
the most likely hidden state sequence is:
$\text{argmax}_Q\ P(Q = q_1, q_2, q_3 \mid O) = Sunny, Sunny, Sunny.$

3. Learning

Given an observation sequence O and the set of states in the HMM, learn the HMM parameters $\lambda = (A, B, \pi)$, that is, find $\text{argmax}_{\lambda}\ P(O \mid \lambda)$.

Example: In the weather-mood HMM, what are the parameters (matrices A and B of the HMM) that are most likely to have generated the mood sequence *Happy*, *Sad*, *Happy* ?

A simpler situation:

Consider the case of training a fully visible Markov model. Suppose we had the following set of observation sequences along with the corresponding weather sequences:

Happy	*Sad*	*Happy*
Sunny	*Rainy*	*Sunny*

Happy	*Sad*	*Sad*
Cloudy	*Rainy*	*Sunny*

Sad	*Happy*	*Sad*
Rainy	*Cloudy*	*Rainy*

In this case, we can compute π, A and B using the data as follows:

$$\pi = [P(Q_1 = Sunny) \quad P(Q_1 = Cloudy) \quad P(Q_1 = Rainy)] = \left[\frac{1}{3} \quad \frac{1}{3} \quad \frac{1}{3}\right]$$

	Sunny	**Cloudy**	**Rainy**
Sunny	0	0	1
Cloudy	0	0	1
Rainy	$\frac{2}{3}$	$\frac{1}{3}$	0

	Happy	**Sad**
Sunny	$\frac{2}{3}$	$\frac{1}{3}$
Cloudy	1	0
Rainy	0	1

Actual situation:

We do not actually know the weather sequences.

The **Forward-Backward Algorithm** (also called the **Baum-Welch Algorithm**) gives a way to learn the HMM parameters iteratively.

Expectation-Maximization (EM) Algorithm

- The EM algorithm is an iterative method for finding maximum likelihood estimates of parameters in probabilistic models when the data is incomplete or has hidden (latent) variables.

- It alternates between two steps:
  (i) Expectation (E-step): Estimate the missing or hidden information using the current parameter values.
  (ii) Maximization (M-step): Update the model parameters using these estimates.

  This process repeats until convergence.

- EM is widely used in scenarios like clustering (e.g., Gaussian Mixture Models) and sequence models.

- The Forward-Backward algorithm for Hidden Markov Models (HMMs) is a special case of EM algorithm, where the E-step involves computing state probabilities using dynamic programming, and the M-step updates the transition and emission probabilities to maximize the expected complete-data log-likelihood based on those state probabilities.

To understand the Forward-Backward Algorithm, let us first understand the Backward Algorithm for computing the probability of an observation sequence.

For time t and state i, the backward probability $\beta_t(i)$ is defined as follows:

$$\beta_t(i) = P(o_{t+1}, o_{t+2}, \dots, o_T \mid q_t = i, \lambda).$$

The Backward Algorithm computes $P(O \mid \lambda)$ for a given observation sequence O in a similar manner to forward algorithm, but **going backwards**.

For $1 \leq t \leq T-1$, we have

$$\beta_t(i) = P(o_{t+1}, o_{t+2}, \dots, o_T \mid q_t = i, \lambda)$$

$$= \sum_{j=1}^{N} P(o_{t+2}, o_{t+3}, \dots, o_T \mid q_{t+1} = j, \lambda)\, a_{ij}\, b_j(o_{t+1})$$

$$= \sum_{j=1}^{N} \beta_{t+1}(j)\, a_{ij}\, b_j(o_{t+1}),$$

which is a recursive equation for $\beta_t(i)$.

Also,

$$P(O \mid \lambda) = \sum_{j=1}^{N} \pi_j \, b_j(o_1) \, P(o_2, o_3, \ldots, o_T \mid q_1 = j, \lambda) = \sum_{j=1}^{N} \pi_j \, b_j(o_1) \, \beta_1(j).$$

Steps of the Backward Algorithm:

1. **Initialization:**

$$\beta_T(i) = 1, \quad 1 \leq i \leq N$$

2. **Recursion**

$$\beta_t(i) = \sum_{j=1}^{N} a_{ij}\, b_j(o_{t+1})\, \beta_{t+1}(j), \quad 1 \leq i \leq N, 1 \leq t < T$$

3. **Termination:**

$$P(O|\lambda) = \sum_{j=1}^{N} \pi_j\, b_j(o_1)\, \beta_1(j)$$

Source: Daniel Jurafsky and James H. Martin, "Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models" (3rd edition), 2025. (Online manuscript released August 24, 2025.)
URL: https://web.stanford.edu/~jurafsky/slp3/

The Forward-Backward Algorithm uses the following estimates for the entries of the matrices A and B.

$$\hat{a}_{ij} = \frac{\text{expected number of transitions from state } i \text{ to state } j}{\text{expected number of transitions from state } i}$$

$$\hat{b}_j(v_k) = \frac{\text{expected number of times in state } j \text{ and observing symbol } v_k}{\text{expected number of times in state } j}$$

For a fixed time step t and state j, let us consider the events

A: $o_1, o_2, \ldots, o_t$

B: $o_{t+1}, o_{t+2}, \ldots, o_T$

C: $q_t = j$.

Then we have $\alpha_t(j) = P(A, C \mid \lambda)$ and $\beta_t(j) = P(B \mid C, \lambda)$.

Hence $\alpha_t(j)\,\beta_t(j) = P(A, C \mid \lambda)\,P(B \mid C, \lambda)$.

We have

$P(A, B, C \mid \lambda) = P(A \mid \lambda)\, P(C \mid A, \lambda)\, P(B \mid A, C, \lambda)$ (by chain rule).

As $P(A \mid \lambda)\, P(C \mid A, \lambda) = P(A, C \mid \lambda)$, we have

$P(A, B, C \mid \lambda) = P(A, C \mid \lambda)\, P(B \mid A, C, \lambda)$.

Since B does not depend on A, we have $P(B \mid A, C, \lambda) = P(B \mid C, \lambda)$.

Therefore,

$$P(A, B, C \mid \lambda) = P(A, C \mid \lambda)\, P(B \mid C, \lambda) = \alpha_t(j)\, \beta_t(j).$$

$$\alpha_t(j)\,\beta_t(j) = P(A, B, C \mid \lambda)$$
$$= P(o_1, o_2, \ldots, o_t, o_{t+1}, o_{t+2}, \ldots, o_T, q_t = j \mid \lambda)$$
$$= P(O, q_t = j \mid \lambda).$$

For any fixed time step t,

$$P(O \mid \lambda) = \sum_{j=1}^{N} P(O, q_t = j \mid \lambda) = \sum_{j=1}^{N} \alpha_t(j)\,\beta_t(j)\,.$$

Let $\xi_t(i,j)$ be the probability of being in state i at time t and state j at time $t+1$, given the observation sequence and the model.

$$\xi_t(i,j) = P(q_t = i, q_{t+1} = j \mid O, \lambda).$$

As $P(X \mid Y,Z) = \frac{P(X,Y \mid Z)}{P(Y|Z)}$, we can compute $\xi_t(i,j)$ as follows:

$$\xi_t(i,j) = \frac{P(q_t=i, q_{t+1}=j, O \mid \lambda)}{P(O \mid \lambda)},$$

where

$$P(q_t = i, q_{t+1} = j, O \mid \lambda) = \alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)$$

and

$$P(O \mid \lambda) = \sum_{j=1}^{N} \alpha_t(j)\beta_t(j)$$

Hence the equation for $\xi_t(i,j)$ is:

$$\xi_t(i,j) = \frac{\alpha_t(i)a_{ij}b_j(o_{t+1})\beta_{t+1}(j)}{\sum_{j=1}^{N}\alpha_t(j)\beta_t(j)}.$$

Now, the estimate $\hat{a}_{ij}$ can be computed as:

$$\hat{a}_{ij} = \frac{\sum_{t=1}^{T-1}\xi_t(i,j)}{\sum_{t=1}^{T-1}\sum_{k=1}^{N}\xi_t(i,k)}$$

Let $\gamma_t(j)$ be the probability of being in state j at time t, given the observation sequence and the model.

$$\gamma_t(j) = P(q_t = j \mid O, \lambda).$$

As $P(X \mid Y, Z) = \frac{P(X,Y \mid Z)}{P(Y|Z)}$, we can compute $\gamma_t(j)$ as follows:

$$\gamma_t(j) = \frac{P(q_t=j, O \mid \lambda)}{P(O \mid \lambda)},$$

where

$$P(q_t = j, O \mid \lambda) = \alpha_t(j)\beta_t(j)$$

and

$$P(O \mid \lambda) = \sum_{j=1}^{N} \alpha_t(j)\beta_t(j)$$

Hence the equation for $\gamma_t(j)$ is:

$$\gamma_t(j) = \frac{\alpha_t(j)\beta_t(j)}{\sum_{j=1}^{N} \alpha_t(j)\beta_t(j)}.$$

Now, the estimate $\hat{b}_j(v_k)$ can be computed as:

$$\hat{b}_j(v_k) = \frac{\sum_{t=1\ s.t.o_t=v_k}^{T} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)}$$

Forward-Backward Algorithm (Baum-Welch Algorithm)

function FORWARD-BACKWARD(*observations* of len T, *output vocabulary* V, *hidden state set* Q) **returns** $HMM=(A,B)$

initialize A and B
iterate until convergence
E-step

$$\gamma_t(j) = \frac{\alpha_t(j)\beta_t(j)}{P(O|\lambda)} \quad \forall t \text{ and } j$$

$$\xi_t(i,j) = \frac{\alpha_t(i)\, a_{ij} b_j(o_{t+1})\beta_{t+1}(j)}{P(O|\lambda)} \quad \forall t, i, \text{and } j$$

M-step

$$\hat{a}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \sum_{k=1}^{N} \xi_t(i,k)}$$

$$\hat{b}_j(v_k) = \frac{\sum_{t=1 s.t. O_t=v_k}^{T} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)}$$

return A, B

Source: *Daniel Jurafsky and James H. Martin. 2026. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition. Online manuscript released August 19, 2026. https://web.stanford.edu/~jurafsky/slp3.*

The following graph shows the log-likelihood of the observed sequence *Happy, Sad, Happy* over successive iterations in an implementation of the Baum-Welch (forward-backward) algorithm. The graph shows an increase in the log-likelihood value and its convergence to a stable value, indicating the convergence of the HMM parameter estimates A and B.

